

D2FA流程和API

Created	@September 14, 2024 12:27 PM
No.	27

背景

Passkey有单独的文档，这里基于 ⚡ [Passkey as D2FA 0.14](#) 进行全面的设计（产品和技术）。

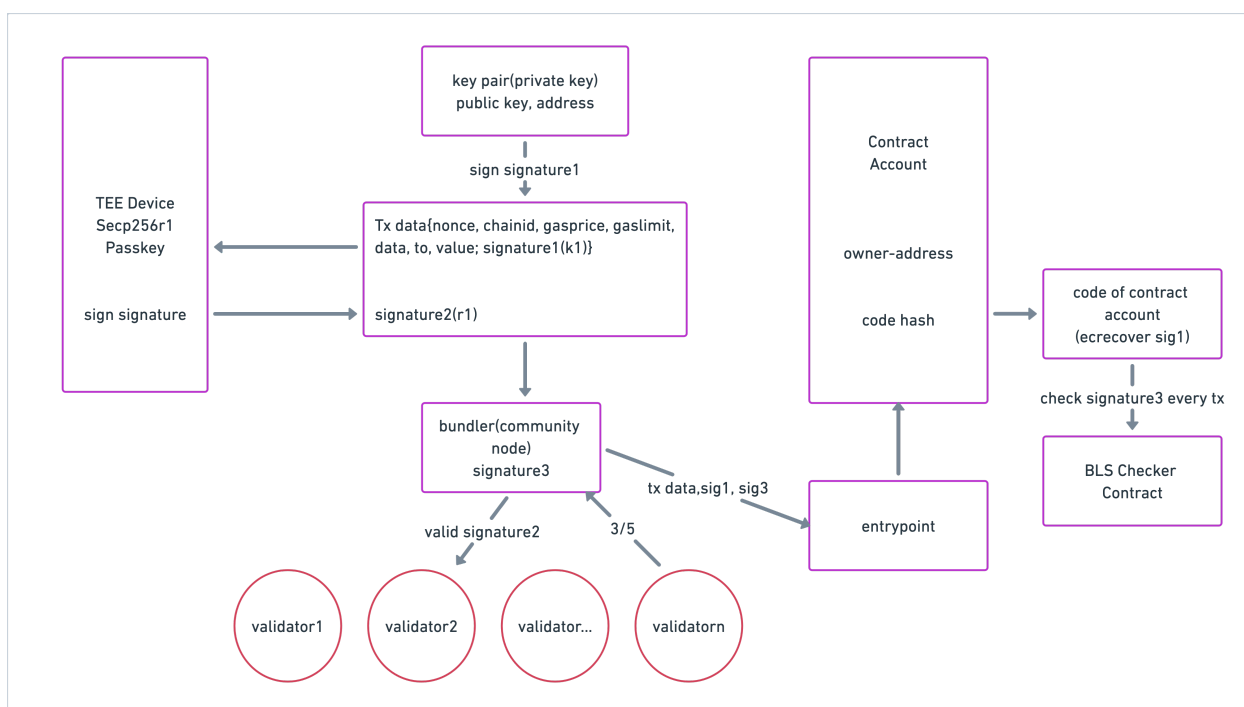


Figure1

流程

1. Community Node的标识：

- 每个community node自己的私钥，目前先EOA(MPC)，使用HD Wallet生成（建议使用服务器上的本地生成根据，然后记录下mnemonic phrase；后面需

要自己也是一个普通AA 账户，然后改造为多签控制私钥更换，就是社区用的AA 账户了。

2. Passkey注册：

- a. Web3 App注册passkey（生成keypair后）：通过API发送给community node public key，做AA账户生成参数，未来可以链上验证（7212），也可以DVT，两种都兼容。
- b. 验证方式是交易参数带 passkey public key，然后链上verify；或者联系 validator verify。

3. 生成Web2等的AA Account

- a. airaccount生成一个local key pair做account的private key，key本身使用RSA 非对称加密（community node开始的标识，private key）
- b. 如下图，Web2等外部非Web3账户，会在创建后在内部创建绑定记录，无感无缝拥有AA Account。
- c. passkey的public key作为参数参与账户创建？（看合约设计）
- d. 签名则需要解密private后进行交易签名，这个是一个原子动作：解密+签名。
- e. 未来每个用户私钥的操作，都做审计记录，基于TEE，还有外部monitor做监控。

4. Web3账户的绑定

- a. 已经创建的AA和EOA
- b. 简单说EOA基于7702,使用我们的模板代码，则就是我们AirAccount（临时或永久）
- c. AA的需要原来的合约账户支持模块化，然后安装我们的模块；或者授权我们的安全合约对交易进行再次审核和过滤。
- d. 只使用我们的paymaster，在做gas sponsor的同时，对交易做安全合约过滤，也是一个思路
 - i. 但这个操作，如果私钥丢失，资产会丢失，因为没有深度绑定的D2FA和 social recovery

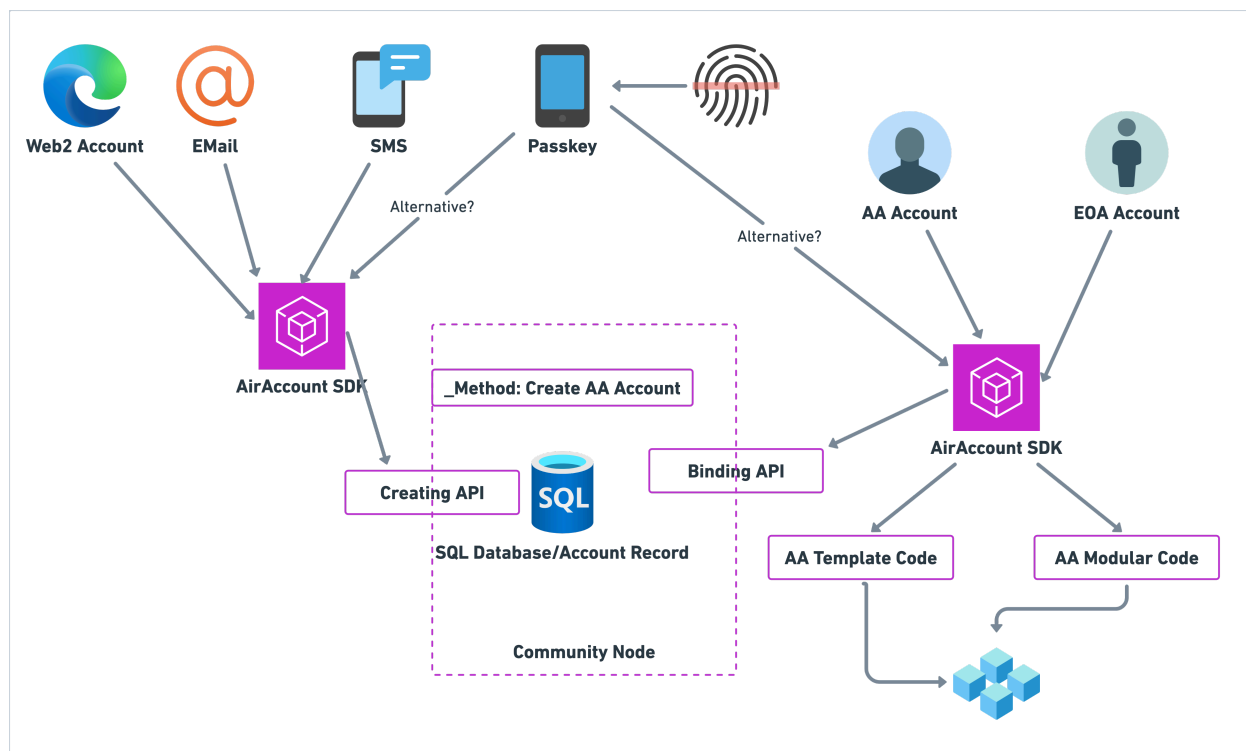
5. Airdrop操作

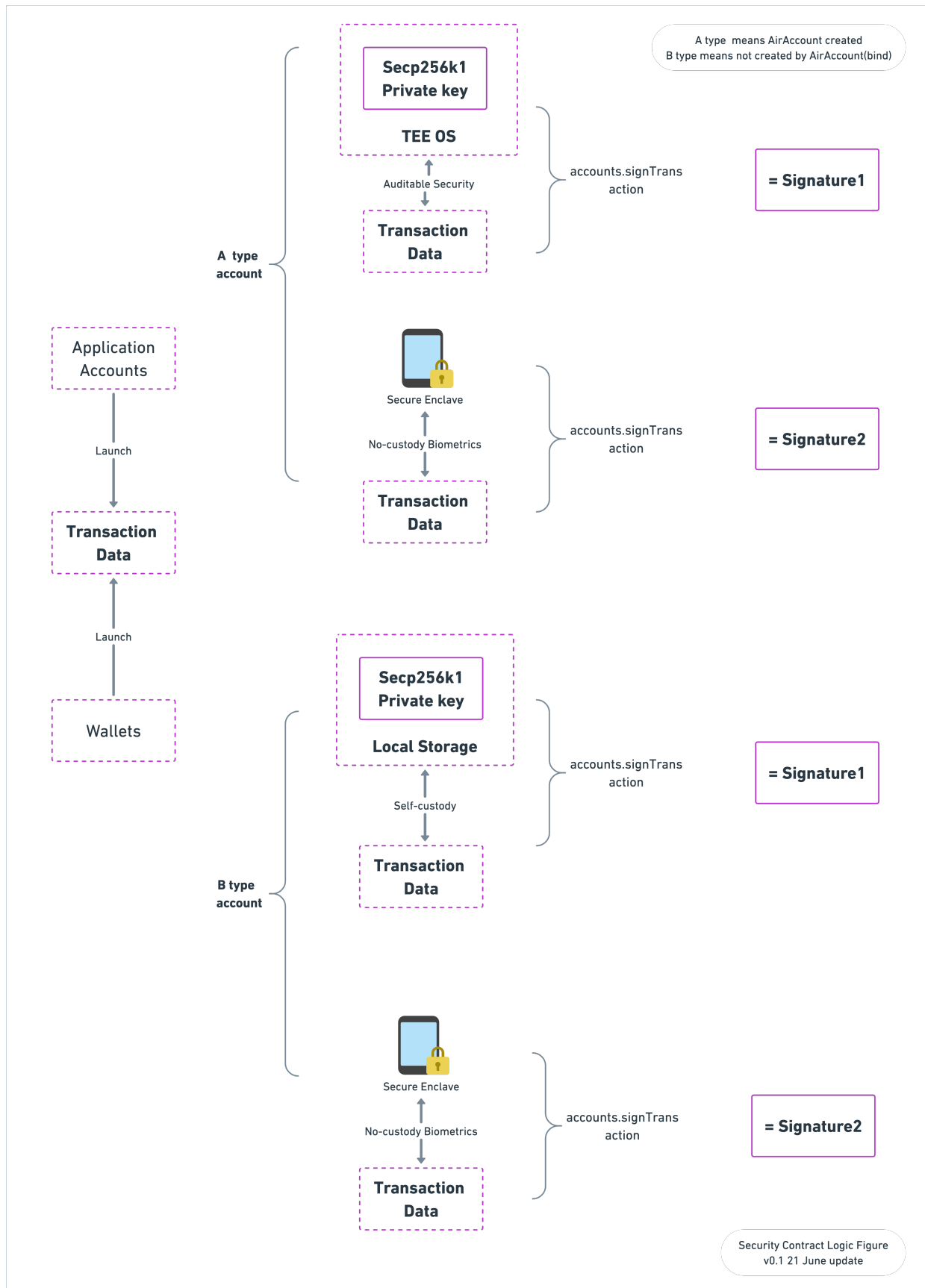
- a. 在生成AA Account后，一般会提供一些参数，例如哪个社区，空投哪个NFT，年卡和预充值多少
- b. 更多业务都可以设计（嵌入游戏的免gas card，免费领取）
- c.

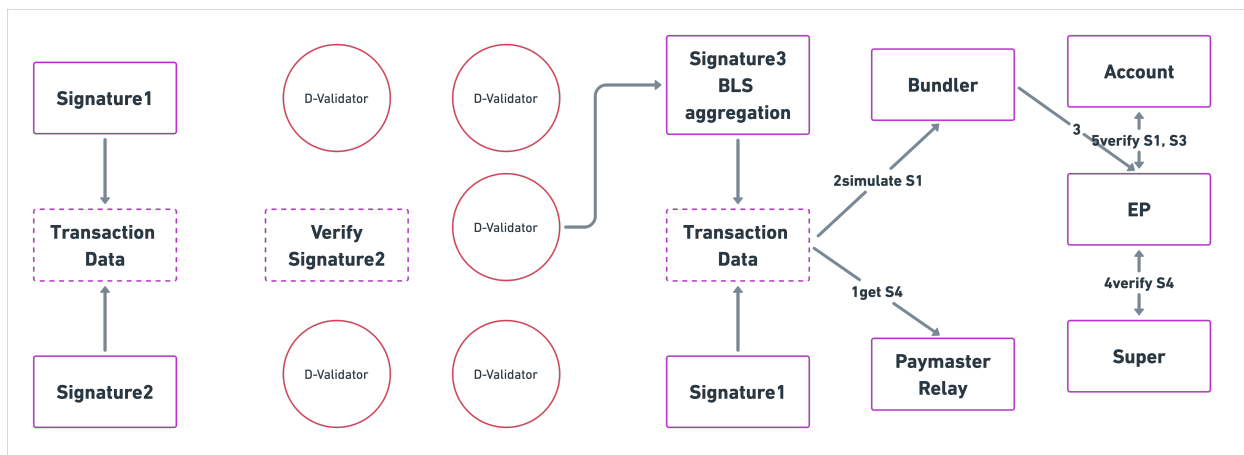
6. Validtor

- a. 任何交易都有两个签名（参考上面passkey文档）
- b. D2FA功能是对所有的交易进行一次r1签名verification
- c. 完成后BLS+TSS聚合签名（第三个签名）
- d. 而所有AirAccount创建的合约账户，都要对交易进行二次签名验证，对聚合签名验证通过后，才提交交易，因为内置于合约，则无指纹无法通过validator验证，提升了安全性
- e. 参考下图

创建和绑定、签名验证



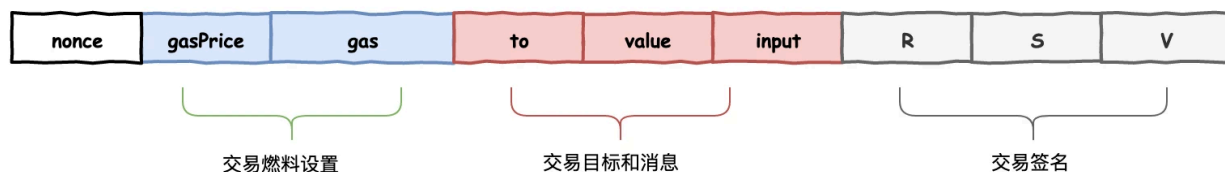




Passkey交易签名和验证

交易数据(可以单独开一章了)

常规以太坊交易数据（这里面少了calldata，另外关于gas有eip改进）：



以太坊技术与实现

目前遵守4337的userops数据规范，而userops可以是一个tx数组（看如何定义？）。Entrypoint有两个方法：`handleOps`，`handleAggregatedOps`，对应了多个tx的userop。之所以不叫Tx，是因为有自己的其他逻辑，例如gas 支付，例如聚合签名等等。

英文定义：Useroperation data structure that describes a transaction to be sent on behalf of a user. It is not an actual Blockchain transaction but is similar to it as it contains fields like "sender", "to", "calldata", "nonce", "signature", "maxFeePerGas", "maxPriorityFeePerGas" and more.

数据结构如下：

Normal Tx

```
{
  "sender",
  "to",
  "calldata",
  "nonce",
  "signature",
  "maxFeePerGas",
  "maxPriorityFeePerGas"
}
```

UserOperation

```
{
  address sender;
  uint256 nonce;
  bytes initCode;
  bytes callData;
  uint256 callGasLimit;
  uint256 verificationGasLimit;
  uint256 preVerificationGas;
  uint256 maxFeePerGas;
  uint256 maxPriorityFeePerGas;
  bytes paymasterAndData;
  bytes signature;
}
```

目前大部分userOp都是一个交易，少部分是2-3,极少部分是多个交易
因此，calldata部分，多数时候实际上就对应了Tx的calldata？

Structure of UserOperation

Field	Description
address sender	Smart Contract Wallet address
uint256 nonce	Nonce value verified in EntryPoint to avoid replay attacks. SCW is not expected to implement this replay prevention mechanism.

bytes initCode	Bytes containing calldata for SCW Factory contract. First 20 bytes are Factory contract address and rest is calldata of function to be called on Factory Contract.
bytes callData	Calldata of function to be executed on SCW. It can be any function on SCW (eg execute or executeBatch) which usually then further calls a dApp smart contract. It can even call other methods of SCW internally as well. It's upto you how you implement this method in SCW.
uint256 callGasLimit	Gas limit used while calling the SCW method from EntryPoint contract using 'callData' above.
uint256 verificationGasLimit	This value is used for multiple purposes. It is used as gas limit while calling SCW Factory contract, calling verification methods on SCW and Paymaster and calling postOp method on Paymaster. In short this is gas limit used in calling verification methods on SCW and Paymaster along with postOp method on Paymaster. To be more precise, on top of it, there are other lines in EP whose gas used is accounted in verificationGasLimit. For now if you don't understand, don't worry it'll be explained later in this article how its value is calculated while we are going through the EntryPoint.
uint256 preVerificationGas	This field is also critical to understand properly. In short, bundler can make profit using this field, if used properly. This is the gas counted on EP as a part of transaction execution which can't be tracked on chain using gasleft() opcode. This value contains Base Gas Fee (21000) + calldata cost of handleOps() method + any logic on-chain for which gas can't be tracked (eg. lines of code after calculating refund for bundler) Don't worry if you don't get the above explanation, we'll explain this field in detail later.
uint256 maxFeePerGas	This is the max fee per unit of gas that UserOp is willing to pay. It is similar to how maxFeePerGas is defined in EIP-1559 for gas calculation of ethereum transaction. EntryPoint uses this field to calculate gasPrice for the refund calculation to bundler.

	$\text{gasPrice} \Rightarrow \min(\text{maxFeePerGas}, \text{maxPriorityFeePerGas} + \text{block.basefee})$ Do you know? One of the responsibilities of EntryPoint is to make sure bundler gets the gas refund for the transaction execution. Either paymaster pays for it or SCW itself pays the gas refund.
uint256 maxPriorityFeePerGas	This is max priority fee per gas that UserOp is willing to pay. It is similar to how maxPriorityFeePerGas is defined in EIP-1559 for gas calculation of ethereum transaction. $\text{gasPrice} \Rightarrow \min(\text{maxFeePerGas}, \text{maxPriorityFeePerGas} + \text{block.basefee})$ Do you know? EntryPoint always refund the bundler for gas in native currency of the blockchain.
bytes paymasterAndData	It contains bytes representing paymaster related information. First 20 bytes is paymaster address and rest represents data to be used by Paymaster for verification. It is empty, if paymaster is not used to sponsor the transaction for given UserOp.
bytes signature	It represent the data to be passed to SCW for verification purpose. Usually it's the signature of userOpHash signed by the owner of SCW but it can be utilised in other ways also.

本地passkey签名

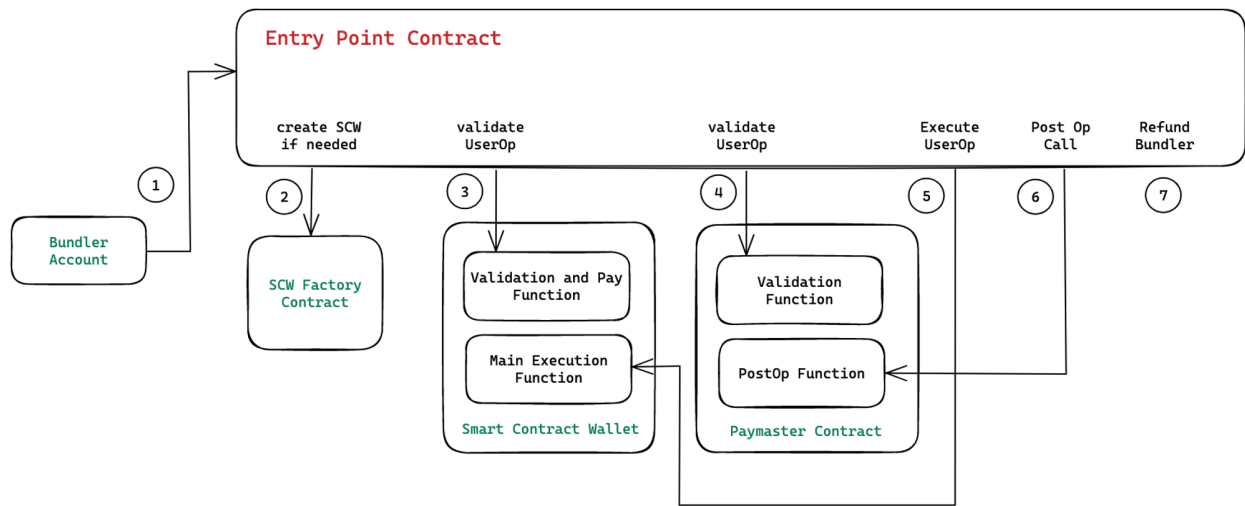
从Figure1可以看到，对于交易数据，需要提交给本地手机，popup biometrics验证，指纹或者其他方式验证通过后，调用本地secure enclave的能力，对Tx data进行签名。

参考黑客松作品：

时序图

ERC4337版

自己画一遍



4337+D2FA版

增加Signature2和Signature3的过程，包括链上和链下

EIP7702版

不同账户类型的抽象要一致

RIP7560版

Native版本也要抽象一致